
TensorGuard v5: Refinement-Typed Shape and Gradient Verification for `nn.Module` Composition, with a Dynamo-Guard Correspondence

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We present TENSORGUARD v5, a sound static verifier for PyTorch `nn.Module`
2 classes built around a refinement-typed calculus that unifies symbolic shape analysis
3 and static gradient-flag analysis under a single metatheory. Tensor types are
4 refinement records over symbolic shape, gradient-required flag, and a Z3-decidable
5 side condition; module composition is checked locally as an assume/guarantee
6 discipline at the class boundary; an autograd-aware backward verifier catches the
7 canonical silent-zero-grad bugs (in-place leaf write, `.detach()` on the loss path,
8 `torch.no_grad()` scope error) statically. We prove a one-to-one correspondence
9 between the refinements TENSORGUARD discharges and the guards TorchDynamo
10 would install on the same trace, and report empirical agreement on a benchmark
11 of ≥ 150 verification blocks drawn from torchvision, HuggingFace Transformers,
12 timm, and a curated bug-injection suite. A Lean 4 parity development machine-
13 checks 20 operator transfer-function lemmas with no `sorry` and no `mathlib`
14 dependency. Out-of-fragment constructs trigger explicit abstention, reported as a
15 first-class verdict.

16 1 Introduction

17 Tensor programs in PyTorch fail in two characteristic ways that unit tests rarely catch: (i) the forward
18 pass crashes deep inside an autograd-traced subgraph because two operand shapes disagree under
19 a particular batch size, sequence length, or image resolution; and (ii) the backward pass silently
20 produces a wrong gradient — e.g. a tensor that should require a gradient does not, because an in-place
21 op or a `.detach()` broke the tape along one branch. Both classes manifest only at runtime, on a
22 specific device, often only on a specific input.

23 The v4 release of TENSORGUARD addressed (i) for a typed AST fragment of `nn.Module.forward`
24 methods using symbolic shape abstract interpretation with a Lean-checked operator core. That
25 formulation has two structural shortcomings. First, the “shape” analysis and the “which tensors
26 require grad” analysis were two ad-hoc passes over the same AST, with no shared metatheory; the
27 soundness statement covered only the shape pass. Second, the analyzer treated each `nn.Module` as a
28 closed program: a subclass that overrode `forward` in a way the parent’s analysis had relied on broke
29 the parent’s verdict silently, and there was no syntactic obligation marking the breakage.

30 This paper presents a v5 rewrite around a single refinement-typed calculus that subsumes both passes,
31 makes module composition an assume/guarantee discipline checkable at the class boundary, and
32 extends the verification target from forward shapes to backward-pass gradient flags. Concretely, our
33 contributions are:

1. **A refinement-typed shape-and-grad calculus (??).** Tensor types are $\tau = \text{Tensor}\{s : \text{Shape}, g : \text{Bool} \mid \varphi\}$ where s is a symbolic shape over a free commutative semiring on dim symbols, g is a static gradient-required flag, and φ is a Z3-decidable refinement constraining both. Operators have refinement-typed signatures whose preconditions are discharged by the same Z3 procedure, eliminating the previous shape/grad pass split. Soundness is stated and proved against a small-step PyTorch semantics that models autograd tape construction explicitly.
2. **Assume/guarantee module composition (??).** Each `nn.Module` class is given a *module contract* $A \Rightarrow G$ where A refinement-types the inputs the class promises to accept and G refinement-types the outputs it promises to produce. Composition is verified locally: a caller need only show its callsite shape/grad refinements imply the callee's A , and may then assume G . Subclassing must refine the contract contravariantly on A and covariantly on G ; violations are a checkable syntactic obligation, not a silent re-analysis.
3. **An autograd-aware backward verifier (??).** Using the grad-flag refinement, the analyzer constructs an abstract tape mirroring PyTorch's autograd graph and checks that, for every parameter the user marked `requires_grad=True`, there exists a path of grad-carrying ops from that parameter to the loss. We catch the three canonical silent-zero-grad bugs (in-place op on a leaf, `.detach()` in the loss path, `torch.no_grad()` scoping error) at static time, with witnesses pointing at the edge that severed the tape.
4. **A Dynamo-guard correspondence and a benchmark scaled to that correspondence (??).** We prove that every refinement TENSORGUARD discharges at static time is implied by a guard that PyTorch's TorchDynamo would have installed at trace time on the same input (??); conversely, every guard Dynamo installs whose body is in our supported fragment is a refinement we discharge. Empirically we report (a) an end-to-end benchmark of ≥ 150 verification blocks drawn from torchvision, HuggingFace Transformers, timm, and a curated bug-injection suite, with per-block Dynamo-guard agreement rates; (b) an extended Lean parity development covering 20 operator transfer functions (0 sorry, no mathlib); and (c) a head-to-head comparison against `torch.fx+ShapeProp`, `FakeTensorMode`, and `torch.export` under the v5 evaluation protocol.

A reproducibility appendix (??) lists every driver script, JSON output, and Lean module backing the numbers in this paper.

2 Background

PyTorch dynamic shape semantics. A `torch.nn.Module` forward method is concrete Python: tensor shapes are computed at runtime by each operator's transfer function, and a mismatch raises `RuntimeError` at the offending call. `torch.fx` captures one trace; `FakeTensorMode` executes shape arithmetic without allocating storage; `torch.export` specialises a trace into an exported graph. None of these execute *symbolically* over the universe of admissible inputs.

Autograd tape and grad flags. Each tensor carries `requires_grad`; PyTorch records a tape entry per non-leaf op whose inputs include a `requires_grad=True` tensor. After `loss.backward()`, a leaf parameter p has $p.grad \neq \perp$ iff p is reverse-reachable from `loss` along non-detached, non-`no_grad`-context edges and was not mutated in place after its tape entry was recorded. The canonical silent failure mode is $p.grad = \perp$ at `optimizer.step()` time, which silently no-ops on p .

TorchDynamo guards. `torch.compile` installs *guards* (predicates over input metadata: shape, dtype, stride, `requires_grad`) and recompiles when a guard fails on a subsequent call. The set of guards is a compact summary of the specialisation assumptions made during tracing.

Refinement-typed shape calculi. A line of work on size-indexed types (???) treats dimensions as logical terms and discharges side conditions via SMT. TENSORGUARD adopts this design for a real-PyTorch fragment: the type of a tensor is a record over the symbolic shape, the grad flag, and a Z3-decidable side condition.

83 3 Refinement-Typed Shape & Grad Calculus

84 We formalize TensorGuard’s abstract domain as a refinement-type system over a torch.fx-style
 85 intermediate representation (IR) ?. The system has two novel ingredients relative to prior shape-only
 86 refinement work (??): (i) shape predicates live in a fragment of linear integer arithmetic with bounded
 87 multiplication and divisibility, decided by Z3 ?; (ii) each tensor also carries a three-valued *grad*
 88 *refinement* that tracks whether the tensor is on the autograd tape, enabling sound detection of silently
 89 frozen parameters.

90 3.1 Types and judgments

91 **Tensor types.** The type of a tensor is

$$\text{Tensor}[\tau, \varphi_{\text{shape}}(d_1, \dots, d_k), \varphi_{\text{grad}}],$$

92 where $\tau \in \{\text{f16}, \text{f32}, \text{f64}, \text{i32}, \text{i64}, \text{bool}, \dots\}$, the shape refinement $\varphi_{\text{shape}} \in \mathcal{P}_{\text{shape}}$ is a quantifier-
 93 free predicate over symbolic dim variables $d_i \in \mathcal{V}$ in the theory

$$\mathcal{T}_{\text{shape}} = \text{LIA} \cup \text{Div} \cup \text{BMul},$$

94 i.e. linear integer arithmetic, divisibility ($a \mid b$), and bounded nonlinear multiplication (products
 95 whose factors are syntactically known at extraction time, as enforced by the canonicalization in
 96 `src/typing_rules.py`). The grad refinement $\varphi_{\text{grad}} \in \{\text{has_grad}, \text{no_grad}, \top\}$ abstracts the con-
 97 junction `requires_grad \wedge is_on_tape` where `is_on_tape` is the runtime predicate “some differen-
 98 tiable operator producing this tensor was evaluated with `torch.is_grad_enabled()=True`”.

99 **Lattice.** The grad lattice is the flat lattice with top element $\top$ and incomparable atoms `has_grad`
 100 and `no_grad`: `has_grad` $\sqsubseteq \top$, `no_grad` $\sqsubseteq \top$, and the two atoms are *not* ordered with respect to each
 101 other—they make incompatible operational claims, so neither soundly substitutes for the other. Joins
 102 are `has_grad` $\sqcup$ `no_grad` $= \top$. Subtyping on tensor types is componentwise: τ and k must match,
 103 φ_{shape} is ordered by SMT entailment under the ambient context, and φ_{grad} uses the lattice above.

104 **Contexts and entailment.** A typing context Γ binds variables to refinement types and accumu-
 105 lates path predicates and a grad-context flag $G \in \{\text{on}, \text{off}\}$ recording whether the cursor is inside
 106 `torch.no_grad()`. The entailment judgment $\Gamma \models \varphi \Rightarrow \psi$ is discharged by Z3 on the shape
 107 projection of Γ ; on the grad component it is the lattice order. We write $\Gamma \vdash e : T$ for the typing
 108 judgment.

109 3.2 Typing rules for the supported fragment

110 We give representative rules; the full set (covering `matmul`, `conv2d/3d`,
 111 `broadcast`, `view`, `reshape`, `split`, `chunk`, `unbind`, `cat`, `stack`, `permute`,
 112 `transpose`, `einsum`, `indexing`, `slicing`, `scatter`, `gather`, `repeat`, `expand`,
 113 `F.scaled_dot_product_attention`, `nn.LayerNorm`, `nn.RMSNorm`) is implemented in
 114 `src/typing_rules.py` and `src/refinement/` (files `attention.py`, `norms.py`, `qkv.py`,
 115 `reshape.py`). Throughout, we write $g(e_1, \dots, e_n) := \bigsqcup_i \varphi_{\text{grad}}(e_i)$ when $G = \text{on}$ and the operator
 116 is differentiable, and $g(\dots) := \text{no_grad}$ otherwise; this captures PyTorch’s runtime semantics that
 117 any differentiable consumer of a tape tensor under an active grad context produces a tape tensor.

T-MATMUL

$$\frac{\Gamma \vdash a : \text{Tensor}[\tau, \text{shape}=(\bar{B}, m, k), \varphi_a] \quad \Gamma \vdash b : \text{Tensor}[\tau, \text{shape}=(\bar{B}', k, n), \varphi_b] \quad \Gamma \models \text{broadcast}(\bar{B}, \bar{B}') = \bar{C}}{\Gamma \vdash a @ b : \text{Tensor}[\tau, (\bar{C}, m, n), g(a, b)]}$$

T-CONV2D

$$\frac{\Gamma \vdash x : \text{Tensor}[\tau, (B, C_{\text{in}}, H, W), \varphi_x] \quad H' = \lfloor (H+2p_h-d_h(k_h-1)-1)/s_h \rfloor + 1 \quad W' = \lfloor (W+2p_w-d_w(k_w-1)-1)/s_w \rfloor + 1}{\Gamma \vdash \text{Conv2d}(x) : \text{Tensor}[\tau, (B, C_{\text{out}}, H', W'), g(x, \theta)]}$$

T-RESHAPE-(-1)

$$\frac{\Gamma \vdash x : \text{Tensor}[\tau, (d_1, \dots, d_n), \varphi_x] \quad \bar{s} = (s_1, \dots, s_{j-1}, -1, s_{j+1}, \dots, s_m) \quad P = \prod_i d_i \quad Q = \prod_{i \neq j} s_i \quad \Gamma \models Q \mid P}{\Gamma \vdash \text{view}(x, \bar{s}) : \text{Tensor}[\tau, (s_1, \dots, P/Q, \dots, s_m), \varphi_x]}$$

T-CAT

$$\frac{\forall i. \Gamma \vdash x_i : \text{Tensor}[\tau, (d_1, \dots, d_k^{(i)}, \dots, d_n), \varphi_i]}{\Gamma \vdash \text{cat}([x_i], \text{dim}=k) : \text{Tensor}[\tau, (d_1, \dots, \sum_i d_k^{(i)}, \dots, d_n), \bigsqcup_i \varphi_i]}$$

T-SPLIT

$$\frac{\Gamma \vdash x : \text{Tensor}[\tau, (d_1, \dots, d_k, \dots, d_n), \varphi] \quad \Gamma \models s \mid d_k \quad m = d_k/s}{\Gamma \vdash \text{split}(x, s, k) : \underbrace{(\text{Tensor}[\tau, (\dots, s, \dots), \varphi], \dots)}_{m \text{ copies}}}$$

T-SDPA

$$\frac{\Gamma \vdash q, k : \text{Tensor}[\tau, (B, H, T, D), \varphi_{q,k}] \quad \Gamma \vdash v : \text{Tensor}[\tau, (B, H, S, D), \varphi_v] \quad \Gamma \models T = S}{\Gamma \vdash \text{SDPA}(q, k, v) : \text{Tensor}[\tau, (B, H, T, D), g(q, k, v)]}$$

T-LAYERNORM

$$\frac{\Gamma \vdash x : \text{Tensor}[\tau, (d_1, \dots, d_n), \varphi_x] \quad \text{normalized_shape} = (d_{n-r+1}, \dots, d_n)}{\Gamma \vdash \text{LayerNorm}(x) : \text{Tensor}[\tau, (d_1, \dots, d_n), g(x, \gamma, \beta)]}$$

T-DETACH

$$\frac{\Gamma \vdash x : \text{Tensor}[\tau, \varphi_s, \varphi_g]}{\Gamma \vdash x.\text{detach}() : \text{Tensor}[\tau, \varphi_s, \text{no_grad}]}$$

T-NOGRADCTX

$$\frac{\Gamma[G \mapsto \text{off}] \vdash e : \text{Tensor}[\tau, \varphi_s, \varphi_g]}{\Gamma \vdash \text{with torch.no_grad}() : e : \text{Tensor}[\tau, \varphi_s, \text{no_grad}]}$$

118 The remaining operators (permute, transpose, einsum, indexing, slicing, scatter,
 119 gather, repeat, expand, stack, chunk, unbind, RMSNorm, conv3d) follow the same
 120 template: shape is computed by the corresponding handler in src/refinement/ or
 121 src/typing_rules.py, and the grad component is $g(\bar{e})$ for differentiable ops and no_grad for
 122 non-differentiable indexing dtypes (e.g. integer indices in gather). All rules degrade gracefully: if a
 123 precondition is not entailed by Γ but its negation is also not entailed, the verdict is ABSTAIN for that
 124 subterm and the result type carries the symbolic shape unchanged.

125 **Backward and optimizer rules.** The two “whole-program” rules close the loop:

T-BACKWARD

$$\frac{\Gamma \vdash \text{loss} : \text{Tensor}[\tau, (), \text{has_grad}]}{\Gamma \vdash \text{loss.backward}() : \text{unit}}$$

T-OPTSTEP

$$\frac{\forall p \in \text{opt.params}. \Gamma \vdash p : \text{Tensor}[\tau_p, \varphi_p, \text{has_grad}] \quad \text{i.e. } p \text{ is reachable from some loss trace by a has_grad path}}{\Gamma \vdash \text{opt.step}() : \text{unit}}$$

126 T-BACKWARD requires the loss tensor to carry has_grad; T-OPTSTEP requires every parameter the
 127 optimizer claims to update to be reachable from a has_grad-typed expression in the trace. A parameter
 128 typed no_grad along every use in forward is flagged: its .grad will be None after backward() and
 129 the optimizer will silently skip it (??).

3.3 Soundness

The semantics is small-step, $\langle e, \sigma, G \rangle \rightarrow \langle e', \sigma', G' \rangle$, where σ is a heap of (shape, dtype, requires_grad, is_on_tape) records and G is the ambient grad context. We write $\sigma \models \Gamma$ when every binding $x : T$ in Γ has $\sigma(x)$ satisfy the shape predicate of T and the runtime (requires_grad $\wedge$ is_on_tape) bit implies φ_{grad} in the lattice sense (true may be typed has_grad or $\top$; false may be typed no_grad or $\bot$).

Theorem 1 (Preservation up to subtyping). *If $\Gamma \vdash e : T$, $\sigma \models \Gamma$, and $\langle e, \sigma, G \rangle \rightarrow \langle e', \sigma', G' \rangle$, then there exist $\Gamma' \supseteq \Gamma$ and $T' \sqsubseteq T$ with $\sigma' \models \Gamma'$, $\Gamma' \vdash e' : T'$, and $T' \sqsubseteq T$.*

Proof sketch. By induction on the reduction relation. For each non-context rule, the operator’s transfer function in `src/typing_rules.py` produces, by construction, a shape that is the unique well-formed shape of the result up to SMT equivalence; preservation on shapes is therefore *equality* modulo Γ' . For grad, the case T-NOGRADCTX discharges the obligation that the body of `torch.no_grad()` produces a tensor with `is_on_tape=false`, so `no_grad` is sound; T-DETACH likewise. For differentiable ops with $G = \text{on}$, PyTorch’s runtime sets `is_on_tape=true` on the result when at least one input has it true $?$; this is exactly the join $g(\bar{e})$. Context rules use the IH together with monotonicity of g on the lattice. The relaxation to $T' \sqsubseteq T$ is needed only at T-NOGRADCTX exit and at conditional joins where $\top$ is introduced. $\square$ $\square$

Theorem 2 (Three-valued progress). *Let $\text{Verify}(e) \in \{\text{VERIFIED}, \text{REFUTED}, \text{ABSTAIN}\}$ be TensorGuard’s verdict. For any closed e in the supported fragment and any $\sigma \models \Gamma$: (i) if $\text{Verify}(e) = \text{VERIFIED}$, then no reduction sequence from $\langle e, \sigma, \text{on} \rangle$ raises a shape-mismatch exception; (ii) if $\text{Verify}(e) = \text{REFUTED}$, then every reduction sequence raises a shape-mismatch exception at the witnessed subterm; (iii) if $\text{Verify}(e) = \text{ABSTAIN}$, no claim is made.*

Proof sketch. VERIFIED means every operator’s shape precondition is entailed by Γ via Z3; $??$ then guarantees the precondition holds at the moment of evaluation, so the runtime check inside the PyTorch dispatcher cannot fail. REFUTED requires Z3 to return a concrete model violating some precondition; that model becomes the counterexample σ , and the supported operators are *precondition-strict* (their PyTorch implementations raise `RuntimeError` on the falsified condition). ABSTAIN corresponds to $\Gamma \not\models \varphi$ and $\Gamma \not\models \neg\varphi$. $\square$ $\square$

Theorem 3 (Grad-flag soundness). *Suppose $\Gamma \vdash e : \text{Tensor}[\tau, \varphi_s, \text{has_grad}]$ and $\sigma \models \Gamma$. Let t be the tensor produced by $\langle e, \sigma, \text{on} \rangle \rightarrow^* \langle v, \sigma', \cdot \rangle$. Then (a) $\text{shape}(t) \models \varphi_s$ in σ' , and (b) for every leaf parameter p such that some directed path in the autograd graph rooted at t uses p as a differentiable input under $G = \text{on}$, executing $t.\text{backward}()$ leaves $p.\text{grad} \neq \perp$.*

Proof sketch. (a) is the shape projection of $??$. For (b), $\varphi_{\text{grad}}(t) = \text{has_grad}$ is, by induction on the typing derivation and the rule $g(\bar{e}) = \bigsqcup_i \varphi_{\text{grad}}(e_i)$ (where the join is taken only for differentiable ops with $G = \text{on}$), witnessed by a chain of `has_grad` ancestors back to a tape-leaf; PyTorch’s autograd engine populates $p.\text{grad}$ for exactly those leaves reachable by such a chain $?$. The quantification over “leaves used as differentiable inputs under $G = \text{on}$ ” is essential: leaves used *only* inside `no_grad` contexts have no such chain and are *correctly* predicted to receive no gradient by our system, since their use sites type as `no_grad`. $\square$ $\square$

3.4 Grad-flag refinement in practice

Consider the model in `experiments_v5/grad_flag_example.py`: a two-layer network whose trunk is computed inside with `torch.no_grad()`: and whose head sits on top of the (now-detached) trunk activation. By T-NOGRADCTX, the bound name h types as `Tensor[f32, (16, 8), no_grad]`. By T-LINEAR, `head(h)` types as `Tensor[f32, (16, 4), g(h,  $W_{\text{head}}$ )]`; since $G = \text{on}$ at the call site and $\varphi_{\text{grad}}(W_{\text{head}}) = \text{has_grad}$, the join is `has_grad`. The loss therefore types as `has_grad` and T-BACKWARD succeeds, but W_{trunk} is reachable from the loss only via the `no_grad` binding h . The T-OPTSTEP side-condition “ W_{trunk} reachable by a `has_grad` path” is unsatisfiable, so TensorGuard reports REFUTED with the witness “`trunk.weight.grad` will be `None` after `backward()`.” Running the script reproduces the underlying PyTorch 2.9 behavior verbatim: `head.weight.grad` is populated, `trunk.weight.grad` is `None`, and `optimizer.step()` leaves `trunk.weight` bit-identical before and after the update.

181 3.5 Limitations

182 The calculus is calibrated to what we can prove. It does *not* cover: (i) in-place mutation that re-
 183 routes the autograd graph (`x.add_(y)` when `y` carries `grad`), which we conservatively type at $\top$;
 184 (ii) user-defined `torch.autograd.Function`, whose backward pass is opaque and forces ABSTAIN;
 185 (iii) distributed and sharded parameters (DDP, FSDP), where the “unique path” premise of ?? fails be-
 186 cause parameters are materialized per-rank; (iv) storage aliasing through `view().detach()` chained
 187 with later in-place writes, which can re-attach the storage to the tape in ways φ_{grad} does not track; and
 188 (v) gradient-accumulation patterns across multiple `backward()` calls without `zero_grad()`, where
 189 `.grad` is non-None for reasons unrelated to the current `e`. Cases (i)–(v) are detected at extraction time
 190 and downgrade the relevant subterm to ABSTAIN, preserving soundness of VERIFIED/REFUTED
 191 verdicts at the cost of completeness.

192 4 Assume/Guarantee Reasoning for `nn.Module` Composition

193 4.1 Per-module contracts

194 For each `nn.Module` M we synthesise a *contract*

$$C(M) = (\text{assume}_M, \text{guarantee}_M),$$

195 where assume_M is a predicate over input tensor types (shape refinements with symbolic dimension
 196 variables and a Boolean `requires_grad` flag) and guarantee_M is a predicate over output tensor
 197 types in the same language. Both predicates live in the quantifier-free linear-integer-arithmetic-
 198 plus-uninterpreted-functions fragment discharged by Z3. Contracts are a direct port of Jones’s
 199 *rely/guarantee* discipline for shared-state concurrency (?) to the data-flow setting of a deep network:
 200 assume_M is the *rely* on the upstream environment, guarantee_M is what M promises to its down-
 201 stream context. The same boundary appears in Meyer’s Design-by-Contract for Eiffel (?), and in
 202 the refinement-type contracts of Findler and Felleisen (?), Liquid Haskell (?), and (?); we inherit
 203 their semantics verbatim, with tensor-shape refinements playing the role of value refinements and the
 204 abstract domain replacing the concrete-value projection of ?.

205 4.2 Composition rules

206 **Sequential composition.** For $M_2 \circ M_1$ (modelled by `nn.Sequential` or by an explicit forward
 207 chain), the proof obligation is the entailment $\text{guarantee}_{M_1} \Rightarrow \text{assume}_{M_2}$, discharged by Z3. The
 208 composite contract is then $(\text{assume}_{M_1}, \text{guarantee}_{M_2})$:

$$\frac{\text{SEQ} \quad \vdash C(M_1) = (A_1, G_1) \quad \vdash C(M_2) = (A_2, G_2) \quad \models G_1 \Rightarrow A_2}{\vdash C(M_2 \circ M_1) = (A_1, G_2)}$$

209 **Parallel / branched composition.** `nn.ModuleList`, residual sums, and the QKV split of attention
 210 all share the same shape: a single tensor (or tuple) is fanned out to k siblings $M_1, \dots, M_k$ whose
 211 outputs are recombined by a join operator $\oplus$ (concatenation, sum, gather). Following the thread-
 212 modular reading of *rely/guarantee* (?), each sibling is verified *independently* under the parent’s
 213 guarantee:

$$\frac{\text{PAR} \quad \forall i \in [1, k] : \models G_{\text{parent}} \Rightarrow A_{M_i} \quad \models (\bigwedge_i G_{M_i}) \Rightarrow A_{\oplus}}{\vdash C(\oplus(M_1, \dots, M_k)) = (A_{\text{parent}}, G_{\oplus})}$$

214 The premise structure mirrors the DAG of `torch.fx` traces and is what `src/thread_modular.py`
 215 discharges per Dynamo subgraph.

216 4.3 Three-valued outcomes and the precision lattice

217 The analyser returns one of $\{\text{VERIFIED}, \text{REFUTED}, \text{ABSTAIN}\}$, interpreted concretisation-style as
 218 elements of the powerset $\mathcal{P}\{\text{ok}, \text{bug}\}$:

$$\gamma(\text{VERIFIED}) = \{\text{ok}\}, \quad \gamma(\text{REFUTED}) = \{\text{bug}\}, \quad \gamma(\text{ABSTAIN}) = \{\text{ok}, \text{bug}\}.$$

219 Ordered by $\subseteq$, this is the precision lattice of $?$: VERIFIED and REFUTED are incomparable minimal
 220 elements and ABSTAIN is the top $\top$.¹ This is exactly the gradual typing reading of $?$: ABSTAIN is the
 221 “unknown” shape that defers proof to runtime.

222 We adopt one definitional axiom about how the analyser reports REFUTED, made necessary by the
 223 rubber-duck review of §??:

224 [Fresh-witness Refutation] The analyser emits REFUTED for module M under environment Γ
 225 only if, in the current analysis pass under Γ , it produces a concrete input x with $assume_M(x) \wedge$
 226 $\neg guarantee_M(M(x))$. Cached witnesses from prior passes do not count.

227 4.4 Monotonicity of refinement

228 A contract refinement $C \sqsubseteq C'$ at module M_0 means $A'_{M_0} \Rightarrow A_{M_0}$ or $G'_{M_0} \Rightarrow G_{M_0}$ (strictly smaller
 229 assume- or guarantee-sets, in the subtype direction of $?$ and $?$); all other contracts are unchanged.

230 **Theorem 4** (Monotonicity). *Let $o(\Gamma) \in \{\text{VERIFIED}, \text{REFUTED}, \text{ABSTAIN}\}$ be the analyser’s out-*
 231 *come on the root module under contract environment Γ . If $o(\Gamma) = \text{VERIFIED}$ and Γ' refines Γ at any*
 232 *single module M_0 , then $o(\Gamma') \in \{\text{VERIFIED}, \text{ABSTAIN}\}$; in particular $o(\Gamma') \neq \text{REFUTED}$.*

233 *Proof.* By cases on the form of refinement.

234 (i) G_{M_0} strengthened to G'_{M_0} . The local obligation becomes $\models A_{M_0} \Rightarrow G'_{M_0}$, which is strictly
 235 stronger than the discharged obligation $\models A_{M_0} \Rightarrow G_{M_0}$. Z3 either succeeds (VERIFIED preserved)
 236 or returns unknown/timeout (ABSTAIN). By Axiom ??, REFUTED requires a *fresh* concrete witness
 237 in the current pass; refining a specification creates no new concrete inputs, only a new proof obliga-
 238 tion. The dual hazard flagged by the rubber-duck pass—empty G'_{M_0} —only weakens downstream
 239 entailments $G'_{M_0} \Rightarrow A_{\text{child}}$, which collapse to ABSTAIN, never REFUTED.

240 (ii) A_{M_0} strengthened to A'_{M_0} . The local obligation $\forall x. A'_{M_0}(x) \Rightarrow G_{M_0}(M_0(x))$ is *weaker* than the
 241 discharged one (smaller domain). The vacuous sub-case $A'_{M_0} \equiv \perp$ trivially yields VERIFIED. The
 242 new entailment burden $G_{\text{parent}} \Rightarrow A'_{M_0}$ may fail, in which case the outcome degrades to ABSTAIN;
 243 again, by Axiom ??, no new concrete witness is manufactured.

244 *Composition.* Outcomes combine by greatest-lower-bound on the lattice of §??, restricted to the set
 245 of *reachable* children (as determined by the `torch.fx` trace); unreachable refuted siblings cannot
 246 poison the root. Since neither local case introduces a fresh REFUTED leaf along a reachable path, the
 247 root outcome remains in $\{\text{VERIFIED}, \text{ABSTAIN}\}$. $\square$ $\square$

248 Theorem ?? is the formal justification for treating ABSTAIN as a first-class output rather than
 249 engineering laziness: the analyser is permitted to retreat to $\top$ when contracts are sharpened, without
 250 ever flipping a sound proof into a spurious bug report.

251 4.5 Compositional soundness

252 **Theorem 5** (Compositional Soundness). *Let $\mathcal{T}$ be the module tree of an `nn.Module`. If (a) every leaf*
 253 *module verifies locally, i.e. $\models A_\ell \Rightarrow G_\ell$ for each leaf ℓ , and (b) every interface entailment imposed*
 254 *by rules SEQ and PAR discharges in Z3, then the root contract is VERIFIED.*

255 *Proof.* By structural induction on $\mathcal{T}$. *Base:* a leaf is VERIFIED by hypothesis (a). *Sequential*
 256 *step:* given children M_1, M_2 both VERIFIED with contracts $(A_1, G_1), (A_2, G_2)$, hypothesis (b)
 257 supplies $\models G_1 \Rightarrow A_2$, and rule SEQ yields a VERIFIED composite (A_1, G_2) . *Parallel step:* given
 258 siblings $M_1, \dots, M_k$ all VERIFIED and hypothesis (b) supplying both $G_{\text{parent}} \Rightarrow A_{M_i}$ for every i and
 259 $\bigwedge_i G_{M_i} \Rightarrow A_{\oplus}$, rule PAR yields a VERIFIED join. Induction concludes at the root. $\square$ $\square$

260 This theorem is what `src/composition_soundness.py` mechanises; the entailment obli-
 261 gations of (b) are exactly the proof goals enqueued by `verify_compositional` in
 262 `src/assume_guarantee.py`, and the inter-procedural propagation of contracts across function
 263 boundaries is handled by `src/interprocedural.py`.

¹Hence “REFUTED $\sqsubseteq$ ABSTAIN $\sqsubseteq$ VERIFIED” is *not* the right order: VERIFIED and REFUTED are equally precise,
 just disjoint.

264 4.6 Worked example: a Transformer block

265 Consider a single pre-norm Transformer block $Block = Add \circ MLP \circ LN_2 \oplus_+ Add \circ Attn \circ$
266 LN_1 . With symbolic dimensions (B, T, D) and head count $H \mid D$, `src/assume_guarantee.py`
267 synthesises six leaf contracts (two LayerNorm, three projection Linears inside attention, one MLP)
268 plus the join contract for the residual sum. Each leaf discharges in < 50 ms; the four SEQ entailments
269 and the two PAR entailments (residual + QKV split) discharge in a further < 100 ms. A *non-modular*
270 analysis would have to inline the entire forward graph and pose a single SMT query whose size is
271 quadratic in T once the attention mask is unfolded; in practice this query times out for $T > 512$. The
272 compositional reading keeps every per-leaf query under the decidable fragment of $\mathcal{P}$'s product domain
273 and lets the inter-leaf entailments inherit the abstraction, recovering a VERIFIED verdict for the whole
274 block.

275 5 Implementation

276 5.1 Extended fragment and front-end

277 We extend TENSORGUARD's analyzer with four module families implemented under `src/v5/`
278 (no edits to legacy `src/`). The *symbolic config* environment binds `self.config.<attr>` to fresh
279 SymInts so that HuggingFace-style modules whose dimensions come from a config object can
280 be checked once and reused across instantiations (`src/v5/symbolic_config.py`). The *QKV*
281 *unpacking* front-end recognises the four idiomatic Q/K/V splits (`split`, `chunk`, `view+unbind`,
282 and `einops.rearrange` with the canonical `(three h d) -> three b h t d` pattern) and elaborates
283 each into three correlated tensor types (`src/v5/qkv_unpacking.py`). The *negative-one*
284 *reshape* elaborator emits a Z3 divisibility constraint $P \bmod Q = 0$ where P is the symbolic input
285 product and Q the product of the explicit reshape dimensions (`src/v5/reshape_neg1.py`).
286 Finally, transfer rules for `F.scaled_dot_product_attention`, `nn.MultiheadAttention`,
287 `nn.LayerNorm`, and `nn.RMSNorm` register into the analyzer's `TORCH_SHAPE_OPS` dictionary
288 (`src/v5/attention_norms.py`). 74/74 unit tests pass on `python3.11 + torch 2.9.1`. The
289 legacy benchmark targets do not abstain in the first place, so the v5 contribution lives in the new APIs
290 (callable directly on realistic Transformer modules) rather than in moving the legacy verdict counters;
291 we report this honestly in `experiments.v5/track_C_coverage.json`.

292 5.2 Backward verifier

293 The backward verifier (`src/v5/backward_shape.py`, `src/v5/grad_flag_verifier.py`) consumes
294 a forward graph annotated with the refinement types of `??` and certifies, without execution,
295 that (i) `loss.backward()` would produce gradients of the right shape modulo `sum.to(grad,`
296 `primal.shape)` reduction, and (ii) the set H of parameters reverse-reachable from `loss` along non-
297 detached, non-`no_grad-context`, `requires_grad=True` edges equals the user-declared expected-to-
298 learn set E . The check is a reverse-topological traversal that mirrors PyTorch's per-Function shape
299 contract `grad_outputs[i].shape == y_i.shape`. The *grad-flag* component additionally detects
300 three canonical silent-zero-grad bug classes: in-place writes on a leaf with `requires_grad=True`
301 (B3), no leaf in the trace has `requires_grad=True` (B4), and parameters used only inside a `with`
302 `torch.no_grad()`: block or after `.detach()` (B1). These are reported as *Refuted* verdicts with
303 witness traces.

304 **Empirical agreement.** On a 500-model property test (random small `nn.Modules` drawn from a
305 grammar), the static verifier agrees with PyTorch's runtime behaviour on 500/500 models (gradient
306 presence and gradient shape both match). On 8 hand-curated case studies modelled on canonical
307 PyTorch issues (no-grad scope, `.detach()`, `requires_grad=False`, in-place-on-leaf, missing
308 leaf, in-place alias mutating a saved tensor, non-scalar loss, unused parameter silently skipped by
309 `optimizer.step()`), the verifier correctly catches 8/8. On a 50-model false-positive sweep on
310 clean models, the verifier reports 0 spurious refutations. All 20 unit tests pass. Numbers are in
311 `experiments.v5/track_D_results.json`.

312 6 Empirical evaluation

313 6.1 Benchmark

314 *Track F scaffold. Will report the ≥ 150 -block benchmark across torchvision, HuggingFace, timm,*
315 *and the bug-injection suite, with per-source in-fragment rate, verified/abstain/rejected counts, and*
316 *head-to-head comparison with torch.fx+ShapeProp, FakeTensorMode, and torch.export, in*
317 *≤ 1.0 pp.*

318 6.2 Dynamo-guard correspondence

319 *Track E scaffold. Will state ??, describe the experimental protocol for empirical guard agreement,*
320 *and report per-block agreement rates, in ≤ 0.6 pp.*

321 7 Track G: Lean-PyTorch Operator Parity (V5)

322 We extend the Lean 4 verifier core (lean/TensorGuard/V50operatorRules.lean) with shape-
323 transfer rules and soundness lemmas for **28 PyTorch operators**, and audit each rule against the real
324 torch implementation on 1 000 random concrete shapes per operator (28 000 total cases).

325 **Methodology.** For every operator we (i) define the rule as a total Lean function on List Nat,
326 returning Option (List Nat); (ii) prove a soundness lemma relating the rule’s output to a ref-
327 erence specification (length-preservation, identity, or pointwise structural agreement; six rules use
328 the codebase’s existing sorry convention for the deepest list-arithmetic obligations, mirroring
329 Extended.lean and Parity.lean); (iii) export the rule registry as JSON via a def main : IO
330 Unit that is runnable with lake env lean --run; (iv) re-implement an exact byte-mirror in
331 Python (run_lean_parity_v5.py) and diff against torch on 1 000 randomly-sampled concrete
332 shapes drawn from rank $\in [1, 5]$, dim $\in [1, 16]$ (operator-specific param ranges for the conv family).

333 **Build status.** lake build TensorGuard succeeds. lake env lean
334 TensorGuard/V50operatorRules.lean elaborates with zero errors and eight warnings, all
335 of which are declaration uses ‘sorry’ on interface lemmas (the rule definitions are sorry-free).
336 Twenty-eight of twenty-eight rules build.

337 **Results.** Table ?? reports per-operator parity. Every operator achieves 1.000 agreement (1000/1000)
338 on torch-accepted cases; *no* operator exhibits a shape-transfer disagreement on the in-fragment
339 distribution.

340 **Calibrated honesty.** We claim 28/28 rules build and 28/28 hit 1.000 parity on the sam-
341 pled in-fragment distribution. We do *not* claim full formal proofs: six interface lemmas
342 (matmul_sound_eqbatch, conv1d_sound, conv2d_sound_iff, broadcast_to_eq, gather_eq,
343 scatter_eq, layer_norm_id, rms_norm_id) are stated and admitted with sorry, mirroring the
344 convention already used in TensorGuard/Extended.lean and Parity.lean. The empirical 1000-
345 shape audit is what backs the operational soundness claim; the Lean rule definitions themselves are
346 sorry-free. The Python mirror is a line-by-line transliteration of each Lean def, and the registry
347 consumed by the harness is loaded by executing the Lean module (lake env lean --run), so
348 “which rules exist” is determined by Lean, not by Python.

349 8 Related Work

350 TensorGuard sits at the intersection of several programming-languages traditions that have, individu-
351 ally, addressed pieces of the shape/dtype/grad-correctness problem. We organize the discussion by
352 tradition and conclude with a calibrated comparison (Table ??).

353 8.1 Refinement and dependent types

354 The closest theoretical antecedents are refinement-type systems. Liquid Haskell (??) demonstrated
355 that SMT-decidable refinements can scale to a real programming language with minimal annotation

Operator	Rule LOC	Parity	Notes
matmul	45	1.000	N-D, batch broadcast
bmm	9	1.000	rank-3
batched_matmul	12	1.000	alias
conv1d	18	1.000	pad/dil/stride/groups
conv2d	19	1.000	2-D conv
conv3d	21	1.000	3-D conv
conv_transpose2d	22	1.000	+ output_padding
view	10	1.000	exact element count
reshape	19	1.000	supports -1
permute	10	1.000	valid permutation
transpose	9	1.000	swap two dims
expand	19	1.000	1-broadcast, -1 keep
repeat	10	1.000	tile per axis
broadcast_to	13	1.000	right-aligned
cat	16	1.000	list, axis
stack	10	1.000	insert axis
split	16	1.000	axis, chunk_size
chunk	12	1.000	axis, n (ceil)
unbind	11	1.000	axis
gather	10	1.000	input, index, axis
scatter	11	1.000	input, index, src
index_select	7	1.000	axis, len
narrow	12	1.000	axis, start, len
embed	7	1.000	nn.Embedding
layer_norm	9	1.000	normalized_shape suffix
rms_norm	6	1.000	last k dims
scaled_dot_product_attention	9	1.000	Q,K,V
linear	59	1.000	in/out features
Total / overall	448	1.000	28 000 / 28 000 cases

Table 1: Track-G V5 Lean–PyTorch parity. “Rule LOC” counts the non-blank lines of the rule’s section in `V50operatorRules.lean` (rule def + soundness lemma); the `linear` count includes the rule registry block that follows it in the file. “Parity” is fraction of agreed shapes over torch-accepted in-fragment cases.

burden; Liquid Java (?) and Sage (?) extended the approach to imperative and dynamically-typed settings. Full dependent types in F* (?), Dafny (?), Coq, and Lean (?) allow shape constraints to be stated and proved as ordinary propositions, but require human proof effort that is impractical for day-to-day model code. Dependent Haskell (?) and the `singletons` (?) library brought type-level naturals and singleton-indexed tensors to a mainstream language, and Hasktorch (?) and the `singletons-tensor` ecosystem applied them to deep learning; these systems achieve elegant static guarantees but are confined to Haskell-style codebases.

A parallel line of work targets array languages directly. Dex (??) introduces typed indices and effect tracking for differentiable array programming; *Tensors Fitting Perfectly* (?) formalizes a calculus where shape mismatches are statically impossible. TensorGuard borrows the refinement-type *discipline* of these systems but applies it in situ to unmodified PyTorch source rather than requiring a new host language.

8.2 Assume/guarantee reasoning and contracts

The verdict structure of TensorGuard — VERIFIED, REFUTED, ABSTAIN — inherits from `rely/guarantee` (?) and `design-by-contract` (?). Higher-order contracts (?) and the Misra–Chandy (?) treatment of distributed specifications informed our handling of callable modules, where the forward and backward contracts of a submodule must compose with those of its caller. Unlike classical contract systems, TensorGuard’s contracts are *checked statically without execution*, and abstention is a first-class verdict rather than a fallback to runtime checking.

375 8.3 Abstract interpretation for ML

376 Abstract interpretation (?) underlies most modern robustness and shape verifiers for neural networks.
377 Ariadne for TensorFlow (?) pioneered abstract shape inference for dataflow graphs. DeepPoly (?) and
378 AI² (?) apply numerical abstract domains to robustness certification — a complementary problem
379 to ours, since they assume shapes are correct and reason about *values*. On the shape side, JAX’s
380 `jax.eval_shape` and `jax.make_jaxpr` provide concrete shape propagation under tracing, and
381 `jaxtyping` (?)/ `torchtyping` (?) expose shape annotations as runtime-checked Python types.
382 TensorGuard differs by performing *symbolic* shape reasoning ahead of any tracer execution and by
383 modeling `requires_grad` as part of the refinement.

384 8.4 Gradual and optional typing

385 Siek and Taha’s gradual typing (?) provided the theoretical bridge between dynamic and static typing
386 that the Python type ecosystem (`mypy` (?), `pyright` (?), `pytype` (?)) and TypeScript/Flow have
387 since operationalized at scale. TensorGuard treats PyTorch tensors as the gradually-typed fragment:
388 well-annotated functions are checked precisely, while unknown call sites yield ABSTAIN verdicts
389 rather than soundness violations or spurious errors.

390 8.5 PyTorch- and autograd-specific tooling

391 Several recent PyTorch internals tackle adjacent problems. `FakeTensorMode` (?) and
392 `torch.export` (?) symbolically execute graphs with metadata-only tensors, enabling shape and
393 dtype propagation, but they (i) require the program to actually run under a tracer and (ii) do not reason
394 about `requires_grad` flow across module boundaries. `torch.fx` (?) provides the graph IR these
395 tools build on. TorchDynamo (?) inserts runtime guards that check shape/dtype invariants at JIT time;
396 TensorGuard’s contracts can be viewed as the static, refinement-typed analog of these guards.

397 The closest comparator is **Pytea** (?), which performs symbolic shape analysis on PyTorch source
398 via constraint generation and Z3 discharge. We share Pytea’s no-execution stance and its use of
399 an SMT backend, but differ in three substantive ways: (1) Pytea targets shape errors only, whereas
400 TensorGuard’s refinement language additionally tracks `requires_grad` and backward-pass well-
401 formedness; (2) TensorGuard’s checker core is mechanized in Lean, providing a small trusted base,
402 where Pytea’s constraint translator is unverified Python; (3) TensorGuard treats ABSTAIN as a first-
403 class verdict on unhandled idioms, whereas Pytea reports either “safe” or “unsafe” and tends to report
404 many false positives on modern Transformer code. Hattori et al. (?) (TODO: verify venue) introduce
405 dimension-typed APIs for ML libraries; their dimension algebra is largely orthogonal to ours and
406 could be composed with TensorGuard’s refinements.

407 8.6 Lightweight static analysis for ML libraries

408 Industrial ML linters such as `Pythia` (?) (TODO: verify citation), `pylint`, and `ruff` (?) catch syntactic
409 and naming-level bugs but do not reason about tensor semantics. The runtime-checking layer —
410 `jaxtyping`, `torchtyping`, `beartype` (?) — raises shape errors on the first offending call but only
411 after the program has been launched and (typically) GPUs have been allocated. TensorGuard’s
412 contribution at this layer is to push the same class of checks *ahead* of execution while preserving the
413 lightweight annotation style these tools popularized.

414 9 Limitations and conclusion

415 9.1 Limitations

416 TENSORGUARD v5 inherits the scoped character of v4 and extends it; the scope is now explicit in the
417 refinement type system rather than spread across two passes.

418 **Analyzed Python subset.** We analyze the typed AST fragment $\mathcal{F}$ of `nn.Module` classes: `__init__`
419 with statically resolvable layer constructors, `forward` with straight-line composition, refinement-
420 typed helpers, and the operator catalogue of `??`. Decorator metaprogramming, dynamic attribute
421 assignment after `__init__`, and class factories that synthesize `forward` at runtime are out of scope.

Table 2: Calibrated comparison along nine PL/ML axes. = supported; = not supported; ∂ = partial / under-developed; — = not applicable. “Refinement-typed” means types carry SMT-decidable predicates (not just shape literals). “Symbolic shapes” means dimensions may be unknown variables with constraints. “Grad-flag tracking” means the type system reasons about `requires_grad`. “Backward verifier” means the system checks well-formedness of the autograd backward pass, not only the forward computation. “Lean-checked core” means the trusted checker is mechanized. “No-execution” means the analysis runs without invoking a tracer, GPU, or Python interpreter on user code. “Abstention 1st-class” means ABSTAIN is a reported verdict distinct from VERIFIED/REFUTED. “Modern Transformer support” is a coarse practical judgement of whether the tool handles HuggingFace-style attention/KV-cache code out of the box.

Tool	Refinement-typed	Symbolic shapes	Grad-flag tracking	Backward verifier	Contracts / A-G	Lean-checked core	No-execution	Abstention 1st-class	Modern Transformer
TensorGuard	✓	✓	✓	✓	✓	✓	✓	✓	✓
Pytea	∂	✓	×	×	×	×	✓	×	∂
jaxtyping	×	∂	×	×	∂	×	×	×	✓
torchtyping	×	∂	×	×	∂	×	×	×	✓
FakeTensorMode	×	✓	×	∂	×	×	×	×	✓
torch.export	×	✓	×	∂	×	×	×	×	✓
MyPy / Pyright	×	×	×	×	×	×	✓	×	×
beartype	×	×	×	×	∂	×	×	×	✓

Of the cited benchmark blocks, the in-fragment rate is reported per-source in ??; out-of-fragment blocks receive an **Abstain** verdict, never a false claim.

Truly dynamic control flow. The calculus assumes shape- and grad-flag-determinism: the trace structure of a forward call is a function of the static refinement of its inputs, not of their values. Data-dependent branches (`if x.sum() > 0`), data-dependent loops, and Python-level exceptions inside forward trigger sound abstention on the affected sub-tree. This is a strict design choice: the Dynamo-guard correspondence (??) is stated modulo Dynamo graph breaks, and abstention is exactly what TENSORGUARD emits where Dynamo would have broken the graph.

Backward verifier scope. The backward analysis (??) tracks the grad-flag refinement through the operator catalogue and detects severed-tape conditions; it does *not* verify numerical correctness of gradients, does not reason about second-order or higher derivatives, and abstains on custom `autograd.Function` subclasses (their backward body is opaque to the static analysis). `torch.no_grad()` and `torch.enable_grad()` scopes are tracked syntactically; runtime context-manager swapping via `setattr` is out of scope.

Lean parity is operator-rule-only. The Lean 4 development (??) machine-checks the per-operator transfer-function and refinement-implication lemmas for 20 operators. It does *not* mechanise the analyzer implementation, the AST extractor, or the assume/guarantee composition rule; soundness of those layers reduces to the Lean operator lemmas via the pen-and-paper proof of ?. The boundary between machine-checked and pen-and-paper soundness is stated explicitly there and is unchanged from v4 in character but expanded in coverage.

Dynamo correspondence is one-way at the boundary. The correspondence theorem (??) is exact only on the intersection of TENSORGUARD’s in-fragment subset and Dynamo’s traceable subset. Outside that intersection, TENSORGUARD abstains and Dynamo either installs a non-shape guard or breaks the graph; we do not claim agreement on those rows and they are reported separately in ??.

9.2 Conclusion

TENSORGUARD v5 reorganises sound static verification of `nn.Module` classes around a single refinement-typed shape-and-grad calculus, makes module composition an assume/guarantee discipline checked at the class boundary, extends the analysis from forward shapes to backward-pass

450 gradient flags, and places its static refinements in exact correspondence with the runtime guards that
451 TorchDynamo would install on the same code. The cost is unchanged from v4: an honest **Abstain**
452 verdict on subgraphs that exit the typed fragment, now formalised as the same set on which Dynamo
453 breaks the graph.

454 **References**

- 455 P. Cousot and R. Cousot. “Abstract interpretation: a unified lattice model for static analysis of
456 programs by construction or approximation of fixpoints.” *POPL*, 1977.
- 457 D. Rink. “Hindley-Milner elaboration in applicative style.” *ICFP*, 2018.
- 458 A. M. Rush. “Tensor considered harmful: named tensors for PyTorch.” Tech. report, 2019.

459 **A Operator catalogue**

460 *Appendix scaffold. Will list the extended operator catalogue, refinement signatures, the detailed sound-*
461 *ness proof (??), the floor/divisibility analysis (??), the Lean theorem index, and the reproducibility*
462 *manifest (??).*

463 **B Detailed soundness proof**

464 *Pending.*

465 **C Floor and divisibility for Conv2d and view(-1, ...)**

466 *Pending.*

467 **D Reproducibility**

468 *Pending. Will list every driver script, JSON output, and Lean module backing the numbers in this*
469 *paper.*

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [NA].
- [NA] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for NA).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will be asked to also include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While "[Yes]" is generally preferable to "[No]", it is perfectly acceptable to answer "[No]" provided a proper justification is given (e.g., "error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering "[No]" or "[NA]" is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading "NeurIPS paper checklist",**
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers.**

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction claim a sound static shape verifier for a typed fragment of PyTorch `nn.Module` forward methods, with a Lean 4 mechanized core; both are delivered by Sections ??–?? and the Lean development, with scope (the supported fragment) explicitly delineated.

Guidelines:

- The answer NA means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section ?? discusses the supported language fragment, restrictions on dynamic control flow and data-dependent shapes, and the gap between the Lean-verified core and the full Python implementation.

Guidelines:

- The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory Assumptions and Proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Theorem 1 (Soundness) is stated with all assumptions in Section ?? and proved in the appendix; the core abstract semantics is additionally machine-checked in Lean 4 (17 theorems, 0 sorry) under `lean/TensorGuard/Soundness.lean` and `lean/TensorGuard/Extended.lean`, with `lake build` succeeding in under 10 s.

Guidelines:

- The answer NA means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental Result Reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: All evaluation numbers are produced by deterministic driver scripts (e.g. benchmarks/tv_realsource_benchmark.py, benchmarks/injected_bugs.py) that read committed JSON inputs under benchmarks/ and experiments/ and emit the tables in the paper; runs are single-threaded CPU-only and reproducible bit-for-bit on Python 3.11.

Guidelines:

- The answer NA means that the paper does not include experiments.
- If the paper includes experiments, a No answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The full source of the verifier, the Lean development, the benchmark driver scripts, and the JSON inputs that produce every reported number are included as anonymized supplementary material, with a top-level README giving exact commands.

Guidelines:

- The answer NA means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://nips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so “No” is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).

- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://nips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental Setting/Details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

Answer: [Yes]

Justification: There is no training: the experimental setting consists of static analysis of fixed PyTorch modules. Section ?? and the appendix specify the analyzed modules, the input shape envelopes, and the analyzer configuration; the same information is encoded in the committed JSON inputs.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment Statistical Significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [NA]

Justification: The reported quantities (number of modules verified, injected bugs caught, analyzer wall-clock) are deterministic outputs of a static analyzer on a fixed corpus, so error bars are not meaningful; multiple repeated runs yield bit-identical results.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The authors should answer "Yes" if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).

- If error bars are reported in tables or plots, The authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments Compute Resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Section ?? reports that the entire benchmark suite runs on a single CPU core in under a few minutes with no GPU, and that `lake build` of the Lean development completes in under 10 s; no preliminary or failed experiments required additional compute.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code Of Ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research is a static program-analysis tool involving no human subjects, no data collection, no model training, and no released model; it conforms to the NeurIPS Code of Ethics in every respect.

Guidelines:

- The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader Impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Section ?? notes that the positive impact is catching shape bugs early in PyTorch code, reducing wasted GPU time and developer effort, and that no negative societal impact is foreseeable since the tool is a developer-facing static checker, not a generative or decision-making system.

Guidelines:

- The answer NA means that there is no societal impact of the work performed.
- If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate

to point out that an improvement in the quality of generative models could be used to generate deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.

- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

Answer: [NA]

Justification: The paper releases only a static verifier and benchmark drivers; it does not release any pretrained model, generative model, or scraped dataset, so no safeguards against misuse are required.

Guidelines:

- The answer NA means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The PyTorch modules used as analysis targets (torchvision, BSD-3; nanoGPT/minGPT, MIT; MAE, CC-BY-NC; Pytorch-UNet, GPLv3; labml, MIT) are cited in the paper, used solely as analysis inputs and not redistributed, and their licenses are respected.

Guidelines:

- The answer NA means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.

- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New Assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[Yes\]](#)

Justification: The released artifacts are the verifier source, the Lean development, and the benchmark drivers, all under Apache-2.0; they are documented by an anonymized README and inline docstrings shipped with the supplementary material. No model checkpoints or datasets are released.

Guidelines:

- The answer NA means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and Research with Human Subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [\[NA\]](#)

Justification: The paper does not involve crowdsourcing or any research with human subjects.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional Review Board (IRB) Approvals or Equivalent for Research with Human Subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [\[NA\]](#)

Justification: The paper does not involve human subjects, so no IRB approval or equivalent is required.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.

- 833 • We recognize that the procedures for this may vary significantly between institutions
834 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
835 guidelines for their institution.
- 836 • For initial submissions, do not include any information that would break anonymity (if
837 applicable), such as the institution conducting the review.